

Formalizing the Fundamental Theorem of Matrix Product States

1228 lines · 12 modules · 0 sorries · Mathlib v4.27.0 · arXiv:2011.12127

Matrix Product States — A 2-Minute Recap

Setup. Physical index $i \in \{1, \dots, d\}$, bond dimension D .

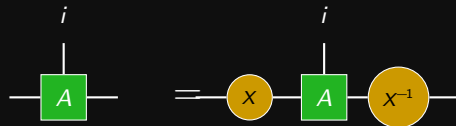
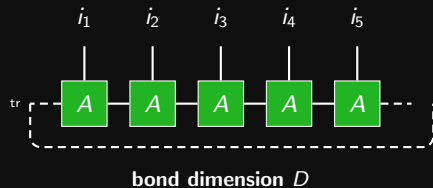
Assign a $D \times D$ matrix A^i to each physical index.

The state (periodic boundary conditions):

$$|V^{(N)}\rangle = \sum_{i_1, \dots, i_N} \text{tr}(A^{i_1} \cdots A^{i_N}) |i_1 \cdots i_N\rangle \quad (1)$$

Gauge freedom. $B^i = XA^iX^{-1}$ with $X \in \text{GL}(D, \mathbb{C})$ leaves every coefficient invariant by cyclicity of trace.

[proved in `GaugeInvariance.lean`]



The Fundamental Theorem of MPS

Theorem (Single-block, injective case)

[arXiv:2011.12127, Thm 4.1]

Let $A = \{A^i\}_{i=1}^d$ be **injective**: the matrices $\{A^i\}$ span the full algebra $M_D(\mathbb{C})$.

If tensors A and B generate the **same MPV family** for all system sizes N , then they are **gauge equivalent**:

$$B^i = X A^i X^{-1} \quad \text{for some } X \in \text{GL}(D, \mathbb{C}) \quad (2)$$

Why does this matter?

- **Classification of gapped phases:** gauge equivalence $\Leftrightarrow$ same phase
- **SPT order:** symmetry action on gauge d.o.f. classifies SPT phases via $H^2(G, U(1))$
- **Renormalization:** injective MPS are fixed points of the RG flow
- **Parent Hamiltonians:** unique ground state iff injective — gauge freedom is the *only* ambiguity

The Full Picture: Canonical Form



Block-diagonal

Each block: A_k^i injective, $\mu_k \in \mathbb{C}$ distinct.

Block MPVs contribute independently:

$$V^{(N)} = \sum_k \mu_k^N \cdot V_k^{(N)} \quad (3)$$

Corollary 4.2: Same state $\Rightarrow$ blocks match pairwise up to gauge + phase.

✓ Single-block theorem

○ Multi-block decomposition

◇ Full corollary

Why Formalize This Proof?

The problem with physics proofs:

- “It is easy to see that. . .”
- Index manipulations by hand
- Implicit assumptions ($D > 0$, fields algebraically closed, . . .)
- Proofs build on each other — errors propagate

This proof specifically:

- Uses deep algebra (Skolem–Noether!)
- Requires careful handling of ring simplicity
- SPT classification *depends* on correctness

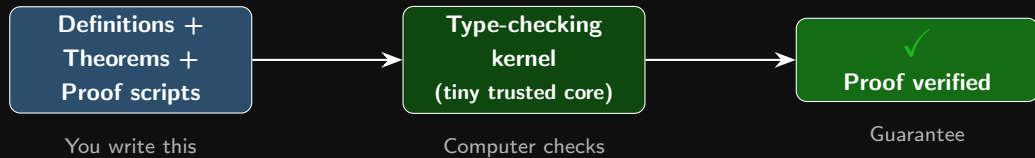
What formalization guarantees:

- Every logical step machine-checked
- All hypotheses made explicit
- No “exercise for the reader”
- **0 sorries** = zero gaps

What we found:

- The paper proof is *correct*
- Many “obvious” steps are genuinely nontrivial
- Precise hypotheses matter: $D > 0$ needed in 6+ places
- The algebraic bypass is essential

What is a Proof Assistant?



Key analogy: A proof assistant is a *compiler for mathematics*.

- Compiler checks *code* for type errors → Proof assistant checks *proofs* for logical errors
- If it compiles, the proof is correct. Period.

What it is NOT:

- Not computer algebra (Mathematica) — those compute, not *prove*
- Not automated theorem proving — the human writes the proof, the machine *verifies*
- Not just “type-checking” — the type system encodes all of logic

Lean 4 and Mathlib

Lean 4 — modern proof assistant (Microsoft Research, 2021+)

- Dependent type theory (CIC)
- Also a general-purpose programming language
- Fast, extensible, growing community

Mathlib — the mathematical library

- > 1 000 000 lines of formalized math
- ~700 contributors
- Covers: linear algebra, ring theory, topology, measure theory, ...
- Crucially for us: **matrix algebras, simple rings, Skolem–Noether**

Recent successes:

- **Liquid Tensor Experiment** (Scholze, 2022)
- **PFR Conjecture** (Tao et al., 2023)
- **Sphere eversion** (2023)

Our project:

Lean 4 + Mathlib v4.27.0

First formalization from
quantum many-body physics

What Does “0 Sorries” Mean?

In Lean, `sorry` is a magic axiom that closes *any* goal:

```
theorem riemann_hypothesis : RiemannHypothesis := by
  sorry -- accepted but flagged as incomplete!
```

With sorries:

- Proof has gaps
- Like `// TODO` in code
- Lean warns: **uses sorry**

With 0 sorries:

- Every step verified by the kernel
- No gaps, no handwaving
- The proof is *complete*

lake build ✓ **Build successful — 0 axioms, 0 sorries**

Compare: a typical proof says “by a standard argument, T is multiplicative.” Our formalization: **159 lines** for multiplicativity alone.

The Challenge: Formalizing Physics Proofs

Physics proof style

- “It follows from the trace”
- Implicit finite-dimensional
- “WLOG assume $D > 0$ ”
- Skip “standard algebra”

Formal proof requires

- Every lemma cited explicitly
- All types/dims declared
- Case split on $D = 0$ vs $D > 0$
- Prove or import from Mathlib

The good news: Mathlib already has an enormous library.

- We get **Skolem–Noether**, **simplicity of $M_n(\mathbb{C})$** , **trace cyclicity**, **Vandermonde determinants** for free
- We had to *build*: trace nondegeneracy, linear extension, multiplicativity, canonical form infrastructure

Expansion factor: ~ 3 pages of paper proof $\longrightarrow$ 1228 lines of Lean $\approx 10\times$ (typical)

Design Choices

Core type:

```
abbrev MPSTensor (d D : ℕ) := Fin d → Matrix (Fin D) (Fin D) ℂ
```

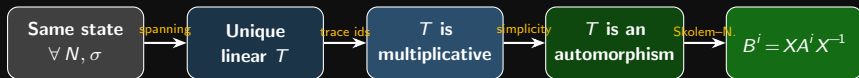
Key definitions:

```
def IsInjective (A : MPSTensor d D) :=  
  Submodule.span ℂ (Set.range A) = ⊤  
  
def SameMPV (A B : MPSTensor d D) :=  
  ∀ N σ, mpv A σ = mpv B σ  
  
def GaugeEquiv (A B : MPSTensor d D) :=  
  ∃ X : GL (Fin D) ℂ,  
    ∀ i, B i = X * A i * X-1
```

Deliberate design decisions:

- **Algebraic injectivity:** $\{A^i\}$ span $M_D(\mathbb{C})$
(avoids CP maps, Perron–Frobenius)
- $GL(D, \mathbb{C})$ for gauge transforms
(Mathlib's GeneralLinearGroup)
- **Sigma-type indices** for block-diagonal
(avoids painful Fin arithmetic)
- **Word evaluation** via lists
(natural recursion)

Proof Strategy at a Glance



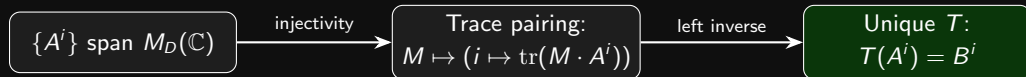
The argument in four sentences:

1. **Same state** $\Rightarrow$ **linear map**. Since $\{A^i\}$ span $M_D(\mathbb{C})$, any linear map is determined by its values on A^i . SameMPV forces a *unique* T with $T(A^i) = B^i$.
2. **Trace identities** $\Rightarrow$ **multiplicativity**. SameMPV gives $\text{tr}(A^i A^j A^k) = \text{tr}(B^i B^j B^k)$. Injectivity of the trace pairing forces $T(MN) = T(M) T(N)$.
3. **Simplicity** $\Rightarrow$ **bijection**. $\ker T$ is a two-sided ideal of $M_D(\mathbb{C})$, which is simple. So $\ker T = 0$ and T is an automorphism.
4. **Skolem–Noether** $\Rightarrow$ **conjugation**. Every automorphism of a matrix algebra is inner: $T(M) = XMX^{-1}$. Apply to each A^i . Done!

Key: Pure algebra — traces, spanning, ring theory.
No wavefunctions, no channels, no spectral theory.

Step 1: Linear Extension

Goal. Since $\{A^i\}$ span $M_D(\mathbb{C})$, there is a *unique* linear map T with $T(A^i) = B^i$.



Key ingredients:

- Trace pairing $\Phi_A : M \mapsto (i \mapsto \text{tr}(M \cdot A^i))$ is injective when A is injective
- SameMPV $\Rightarrow$ Gram compatibility: $\Phi_A \circ \text{lc}_A = \Phi_B \circ \text{lc}_B$
- Range inclusion + finrank argument $\Rightarrow \Phi_B$ also injective
- $T = \Phi_B^{-1} \circ \Phi_A$ is the unique extension

✓ `linearExtension_exists_unique` — 88 lines

Step 2: Multiplicativity — The Key Step

Goal. Show $T(MN) = T(M) T(N)$ for all $M, N \in M_D(\mathbb{C})$.

Strategy: Trace identities from SameMPV, then extend by spanning.

1. **Length-2 traces:** $\text{tr}(A^i A^j) = \text{tr}(B^i B^j)$ from SameMPV
2. **Compatibility:** $\Phi_B \circ T = \Phi_A$ on generators, hence everywhere by spanning
3. **Length-3 traces:** $\text{tr}(A^i A^j A^k) = \text{tr}(B^i B^j B^k)$ from SameMPV
4. **Injectivity of Φ_B :** Conclude $T(A^i \cdot A^j) = B^i B^j$ inject into Φ_B
5. **Bilinear extension:** $\{A^i\}$ span $\Rightarrow T(MN) = T(M)T(N)$ two rounds

The crucial lemma underneath

Trace nondegeneracy: $\text{tr}(MN) = 0 \forall N \implies M = 0$ (31 lines, via single-entry matrices)

✓ `linearExtension_mul` — 54 lines

Step 3: Simplicity $\Rightarrow$ Bijectivity

Have: $T : M_D(\mathbb{C}) \rightarrow M_D(\mathbb{C})$ is a nonzero multiplicative linear map. **Want:** T is bijective (hence an algebra automorphism).



- T multiplicative $\Rightarrow \ker T$ is a two-sided ideal of $M_D(\mathbb{C})$
- $M_D(\mathbb{C})$ is **simple** (no nontrivial two-sided ideals)
- $\ker T \in \{0, M_D(\mathbb{C})\}$; since $T \neq 0$, we get $\ker T = 0$
- Injective + finite-dim $\Rightarrow$ surjective $\Rightarrow$ bijective

[Mathlib]

$T \neq 0$ **proof:** If $T = 0$ then $B^i = 0$, so $\text{tr}(A^i) = 0$ for all i . Since $\{A^i\}$ span, trace vanishes everywhere — but $\text{tr}(\mathbf{1}) = D \neq 0$.

✓ `linear_mul_endomorphism_bijective` — uses `TwoSidedIdeal.ker`, `IsSimpleRing`

Step 4: Skolem–Noether — The Deep Theorem

Skolem–Noether Theorem

Every $\mathbb{C}$ -algebra automorphism of $M_n(\mathbb{C})$ is **inner**: $T(M) = XMX^{-1}$.

In Mathlib:

```
-- Mathlib: every automorphism of End(V) is conjugation
AlgEquiv.eq_linearEquivConjAlgEquiv
```

Our bridge (152 lines in `SkolemNoether.lean`):

1. Promote T to `AlgEquiv` (must prove $T(1) = 1$ from surjectivity!)
2. Transport to `Module.End` via $M_n(\mathbb{C}) \cong \text{End}(\mathbb{C}^n)$
3. Apply Mathlib's Skolem–Noether
4. Convert `LinearEquiv` back to $X \in \text{GL}(D, \mathbb{C})$

Subtlety: $T(1) = 1$ is nontrivial! Pick x with $T(x) = 1$, then
 $1 = T(x) = T(x \cdot 1) = T(x) T(1) = T(1)$.

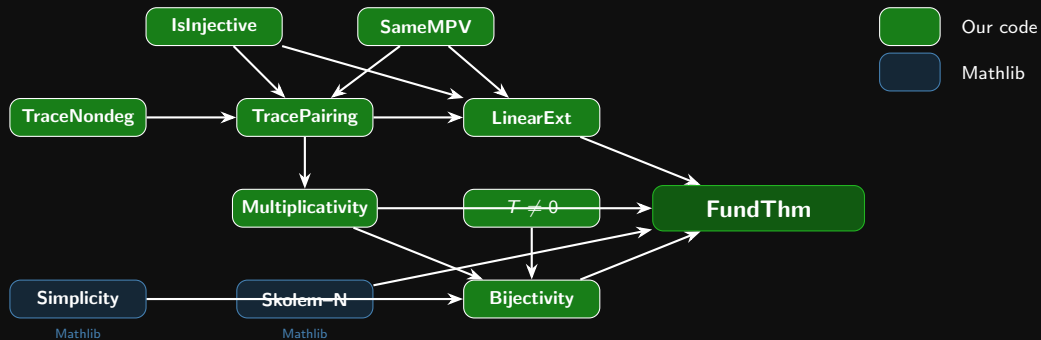
✓ `skolemNoether_matrix` — Mathlib does the heavy lifting

Step 5: Assembly — The Main Theorem

```
theorem fundamentalTheorem_singleBlock
  {A B : MPSTensor d D}
  (hA : IsInjective A) (hAB : SameMPV A B) :
  GaugeEquiv A B := by
classical
cases D with
| zero => exact ⟨1, fun i => by simpa⟩
| succ D' =>
  -- 1. Linear extension
  rcases linearExtension_exists_unique hA hAB with ⟨T, hT, -⟩
  -- 2. Multiplicativity + nonzero => bijective
  have hMul := linearExtension_mul hA hAB hT
  have hBij := linear_mul_endomorphism_bijective T
    hMul (linearExtension_nonzero hA hAB hT)
  -- 3. Skolem-Noether
  let fHom := linearMapToAlgHom T hMul hBij.surjective
  let f := AlgEquiv.ofBijective fHom hBij
  rcases skolemNoether_matrix f with ⟨X, hX⟩
  -- 4. Assemble:  $B \cdot i = T(A \cdot i) = X * A \cdot i * X^{-1}$ 
  refine ⟨X, fun i => ?_⟩
  have hfi : f (A i) = B i := by simpa using hT i
  simpa [hfi] using hX (A i)
```

✓ 78 lines total (FundamentalTheorem.lean) ✓

The Complete Proof Pipeline



What Mathlib Gave Us vs. What We Built

From Mathlib (free!):

- `IsSimpleRing` for $M_n(\mathbb{C})$
- Skolem–Noether
- Trace cyclicity, linearity
- `blockDiagonal'`, block multiplication
- Vandermonde determinant
- $\mathrm{GL}(D, \mathbb{C})$, rank–nullity
- `Submodule.span`, `ext_on_range`

We had to build:

- Trace nondegeneracy (31 lines)
- Trace pairing map Φ_A (99 lines)
- Linear extension existence + uniqueness
- Multiplicativity of T (54 lines)
- $T \neq 0$ argument, $T(1) = 1$
- Matrix $\leftrightarrow$ End transport
- Canonical form, block MPV decomposition
- Vandermonde separation for MPVs

The Algebraic Bypass: What We Deliberately Avoided

Standard physics route

- CP maps / quantum channels
- Perron–Frobenius theorem
- Spectral radius of transfer operator
- Fixed-point equations
- Positivity arguments

Our algebraic route

- Spanning $\Rightarrow$ linear extension
- Trace identities $\Rightarrow$ multiplicativity
- Simple ring $\Rightarrow$ bijectivity
- Skolem–Noether $\Rightarrow$ inner
- Pure linear algebra!

Why the bypass?

- Perron–Frobenius for CP maps is **not in Mathlib**
- The algebraic route uses only linear algebra + ring theory — all well-covered
- Already noted in the paper: injective case doesn't need channel theory
- **Bonus:** the algebraic proof is arguably *cleaner*

Multi-Block: The Canonical Form Structure

```
structure CanonicalForm (d : ℕ) where
  numBlocks      : ℕ
  blockDim       : Fin numBlocks → ℕ
  blockTensor    : (k : Fin numBlocks) → MPSTensor d (blockDim k)
  μ              : Fin numBlocks → ℂ
  block_injective : ∀ k, IsInjective (blockTensor k)
```

Assembly:

```
noncomputable def toTensor : MPSTensor d C.totalDim :=
  fun i => (Matrix.reindex e e)
    (Matrix.blockDiagonal' (fun k => C.μ k • C.blockTensor k i))
```

Key design: Block-diagonal on Σ -type indices $\bigsqcup_k \text{Fin}(D_k)$, then reindex to $\text{Fin}(\sum_k D_k)$.

Uses Mathlib's `blockDiagonal'_mul`, `trace_blockDiagonal'` directly — avoids all `Fin` sub-range arithmetic.

✓ `CanonicalForm.lean` — 52 lines

Multi-Block: The Workhorse Lemma

MPV Decomposition (`mpv_toTensor_eq_sum`)

$$\text{mpv}(\text{toTensor}(C), \sigma) = \sum_{k=1}^r \mu_k^N \cdot \text{mpv}(A_k, \sigma) \quad (4)$$

Proof outline (285 lines in `MultiBlock.lean`):

1. `evalWord` of reindexed tensor = reindex of `evalWord`
2. `evalWord` of block-diagonal = block-diagonal of block `evalWords`
3. With scaling: each block picks up $\mu_k^{|w|}$
4. `trace_reindex`: remove outer reindexing
5. `trace_blockDiagonal'`: split trace into sum
6. Pull out scalar factors

This is the **computational backbone**: reduces multi-block questions to single-block questions weighted by powers of μ_k .

✓ `mpv_toTensor_eq_sum` — proved

Multi-Block: Vandermonde Separation

Setup. Two canonical forms give the same state:

$$\sum_k \mu_k^N \cdot V_k^{(N)}(\sigma) = \sum_j \nu_j^N \cdot W_j^{(N)}(\sigma) \quad \forall N, \sigma$$

Key insight: If $\{\mu_k\}$ are *distinct*, this is a Vandermonde system!

Separation lemma:

Distinct $\mu_k + \sum_k c_k \mu_k^N = 0$ for $N = 0, \dots, r-1$

$\implies c_k = 0 \forall k$

Uses Mathlib's `det_vandermonde`

and `eq_zero_of_forall_pow_sum_mul_pow_eq_zero`.

Extended to MPVs:

If block MPVs are nonzero and μ_k distinct, blocks can be separated.

✓ `vandermonde_separation`, `block_mpv_lin_indep_at_fixed_size` — 162 lines

μ_1^0	μ_2^0	μ_3^0
μ_1^1	μ_2^1	μ_3^1
μ_1^2	μ_2^2	μ_3^2

Vandermonde

$\det \neq 0$ iff μ_k distinct

Multi-Block: What's Next

✓ Single-block fundamental theorem

✓ Canonical form + toTensor construction

✓ MPV decomposition: $V^{(N)} = \sum_k \mu_k^N V_k^{(N)}$

✓ Vandermonde separation of blocks

◦ Block matching: equal sums $\Rightarrow$ pairwise match

◦ Apply single-block theorem per matched pair

◊ Assemble global gauge transform (full Corollary 4.2)

Remaining challenge: Block matching requires showing that blocks from A 's canonical form can be put in bijection with blocks from B 's, with matching μ_k (up to phase).

What We CAN'T Formalize (Yet)

Not in Mathlib (yet):

- Perron–Frobenius for CP maps
- Spectral radius of transfer operator
- Quantum Wielandt (normality)
- Canonical form *existence*

Our approach:

- Axiomatize canonical form as *structure*
- Assume injectivity of blocks
- Assume distinct μ_k
- Prove everything *downstream*

This is sound: we make hypotheses *explicit*. The single-block theorem has **zero** axiomatization — fully proven from Mathlib foundations.

The formalization boundary is set by Mathlib's coverage of functional analysis, not by any fundamental limitation.

What would unlock the rest:

- Perron–Frobenius for positive maps on C^* -algebras
- Spectral theory for finite-dim operators (partially in Mathlib)

Lessons Learned

Surprises (positive):

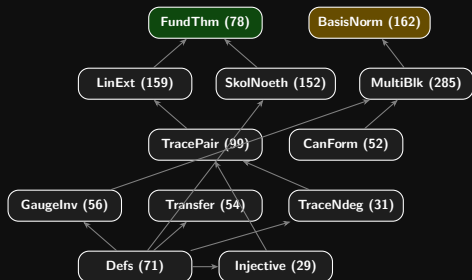
1. Mathlib's abstract algebra is **remarkably complete**
2. The **algebraic bypass** was essential and elegant
3. **Sigma-type indices** for block-diagonal work beautifully
4. Lean 4's tactic mode is pleasant for mathematical reasoning

Key finding: The paper proof is *correct*. Formalization found no errors, but forced precision about every hypothesis (especially $D > 0$).

Challenges:

1. “Obvious” steps ($T \neq 0$, $T(1) = 1$) take real effort
2. **Coercion hell:** $GL \leftrightarrow Matrix \leftrightarrow End$
3. Any `Fin` arithmetic is painful (we avoided it)
4. Expansion factor $\sim 10\times$

Statistics & Module Architecture



Metric	Value
Total lines of Lean 4	1228
Modules	12
Axioms / sorries	0 / 0
Mathlib version	v4.27.0
Expansion factor	$\sim 10\times$

Largest modules:

MultiBlock 285|, BasisNormal 162|,
LinExt 159|, SkolNoeth 152|

Vision: Towards Formal Quantum Many-Body Theory

First machine-checked proof of a foundational theorem in quantum many-body physics

This opens the door to:

- Verified parent Hamiltonians
- Formal SPT classification — $H^2(G, U(1))$
- Verified PEPS theorems — higher-dim TNs
- Formal quantum channels — CP maps

Long-term vision:

- A Mathlib-style library for quantum information
- Bridge algebra/analysis to quantum physics
- Formal verification as **standard tool** in math-phys
- Community effort: contribute back to Mathlib

arXiv:2011.12127

Thm 4.1: **VERIFIED** Cor 4.2: **IN PROGRESS**

The future of mathematical physics is formally verified.

Thank You!

Questions?

`github.com/LionSR/MPSLean`

Built on Lean 4 + Mathlib v4.27.0

Reference: Cirac, Pérez-García, Schuch & Verstraete · `arXiv:2011.12127`

1228 lines · **12 modules** · **0 sorries** · **0 axioms**

Backup: Proving $T(1) = 1$ — The Nontrivial “Obvious” Step

Why is this needed?

To apply Skolem–Noether, T must be an **algebra homomorphism**, requiring $T(1) = 1$.

A multiplicative linear map need not preserve 1!

The argument:

1. T surjective $\Rightarrow$ pick x with $T(x) = 1$
2. $1 \cdot T(1) = T(x) \cdot T(1) = T(x \cdot 1) = T(x) = 1$
3. So $T(1) = 1$ ✓

Crucially uses:

- **Surjectivity** (to find x)
- **Multiplicativity** ($T(x \cdot 1) = T(x)T(1)$)

This is why Step 3 must come *before* Skolem–Noether.

The Lean proof:

```
-- From SkolemNoether.lean
have hOne : T 1 = 1 := by
  obtain ⟨x, hx⟩ := hSurj 1
  calc (T 1 : Matrix (Fin D) (Fin D) ℂ)
    = 1 * T 1      := (one_mul _).symm
    _ = T x * T 1   := by rw [hx]
    _ = T (x * 1)   := (hMul x 1).symm
    _ = T x         := by rw [mul_one]
    _ = 1           := hx
```

Also needed for AlgHom:

```
commutes' := fun c => by
  simp [Algebra.algebraMap_eq_smul_one,
        hOne]
```

Without `map_one'` and `commutes'`, Lean won't build the AlgHom.

Backup: Trace Nondegeneracy — The Foundational Lemma

Statement:

If $\text{tr}(MN) = 0$ for all $N \in M_D(\mathbb{C})$, then $M = 0$.

Proof: Choose $N = E_{ji}$ (single-entry matrix). By cyclicity:

$$\text{tr}(E_{ji} \cdot M) = M_{ij}$$

So $\text{tr}(MN) = 0 \ \forall N$ gives $M_{ij} = 0$ for all i, j .

Where this is used:

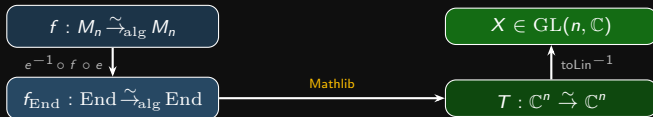
- TracePairing: Φ_A injective
- $\rightarrow$ LinearExtension: T unique
- $\rightarrow$ Multiplicativity of T
- $\rightarrow$ **Everything downstream!**

Full Lean proof (TraceNondeg.lean):

```
theorem trace_mul_right_eq_zero_iff
  (M : Matrix n n ℂ) :
  (∀ N, trace (M * N) = 0) ↔ M = 0 := by
  classical
  constructor
  · intro h
    ext i j
    -- Probe with E_{ji} = Matrix.single j i 1
    have hMN :
      trace (M * Matrix.single j i 1)
        = 0 := h (Matrix.single j i 1)
    -- Use cyclicity: tr(M * E) = tr(E * M)
    have hNM :
      trace (Matrix.single j i 1 * M) = 0 :=
      (trace_mul_comm M
        (Matrix.single j i 1)).symm.trans hMN
    -- Evaluate: tr(E_{ji} * M) = 1 • M_{ij}
    have : (1 : ℂ) • M i j = 0 := by
      simpa [trace_single_mul] using hNM
    simpa using this
  · intro h N
    simp [h]
```

Backup: The Matrix $\leftrightarrow$ End Bridge (“Coercion Hell”)

Problem: Mathlib’s Skolem–Noether works on $\text{End}_{\mathbb{C}}(\mathbb{C}^n)$, not $M_n(\mathbb{C})$. Isomorphic $\neq$ definitionally equal!



Steps 1–3: Transport, apply, convert back:

```
-- Step 1: transport to End
let e := Matrix.toLinAlgEquiv'
let fEnd := e.symm.trans (f.trans e)
-- Step 2: apply Mathlib's Skolem–Noether
obtain ⟨T, hT⟩ :=
  AlgEquiv.eq_linearEquivConjAlgEquiv
    (f := fEnd)
-- Step 3: convert back to GL
let X : GL n C :=
  (GeneralLinearGroup.toLin).symm
    (ofLinearEquiv T)
```

Step 4 (hard): Prove $e(X) = T$ as linear maps:

```
-- Chase: Units.mapEquiv, coe_mapEquiv,
-- GeneralLinearGroup.toLin
have hX_lin :
  e (X : Matrix n n C) = (T : ...) := ...
have hX_lin_inv :
  e ((X⁻¹ : GL n C) : Matrix ..)
    = (T.symm : ...) := ...
```

Step 5: Final calc:

```
calc e (f M)
  = fEnd (e M)           := by simp
  = T o (e M) o T.symm := ...
  = e (X * M * X⁻¹)      := by
    rw [hX_lin, hX_lin_inv]; simp
```

Backup: Injectivity — Algebraic vs. Physical Definitions

Three notions in the MPS literature:

1. **Linear independence** (weakest) — $\{A^i\}$ linearly independent. Requires $d \leq D^2$. *Not sufficient.*

2. **Algebraic injectivity** ← **ours** — $\text{span}_{\mathbb{C}}\{A^i\} = M_D(\mathbb{C})$. Requires $d \geq D^2$.

3. **CP-map injectivity** (physical) — $E_A(X) = \sum_i A^i X (A^i)^\dagger$ has unique full-rank fixed point.

Relationships: (2) $\Rightarrow$ (1): spanning $\Rightarrow$ indep.

(2) $\Rightarrow$ (3): quantum Wielandt. (3) $\Rightarrow$ (2): for **normal** MPS after blocking.

Why we chose (2):

- Purely algebraic: `Submodule.span`
- No CP maps or Perron–Frobenius
- Makes $\ker T$ argument work

The Lean definition:

```
def IsInjective
  (A : MPSTensor d D) :=
  Submodule.span ℂ
    (Set.range A) = ⊤
```

What (3) would need:

- Perron–Frobenius for positive maps
- Spectral theory (partial in Mathlib)
- C^* -algebra structure

We *do* define E_A in `Transfer.lean` and prove positivity, but can't access spectral properties.

Backup: MPV Decomposition — The Full Computation

Goal: $\text{mpv}(C.\text{toTensor}, \sigma) = \sum_{k=1}^r \mu_k^N \cdot \text{mpv}(A_k, \sigma)$ — “one line in physics”, six formal steps:

$$\begin{aligned} \text{mpv}(C, \sigma) &= \text{tr}(\underbrace{C^{i_1} \dots C^{i_N}}_{\text{evalWord}}) && [\text{definition}] \\ &= \text{tr}(\text{reindex}_e(\text{BD}^{i_1} \dots \text{BD}^{i_N})) && [\text{evalWord_reindex}] \\ &= \text{tr}(\text{BD}^{i_1} \dots \text{BD}^{i_N}) && [\text{trace_reindex}] \\ &= \text{tr}(\text{blockDiag}(\mu_k^N A_k^{i_1} \dots A_k^{i_N})) && [\text{evalWord_blockDiag'}_smul] \\ &= \sum_k \text{tr}(\mu_k^N A_k^{i_1} \dots A_k^{i_N}) && [\text{trace_blockDiagonal'}] \\ &= \sum_k \mu_k^N \cdot \text{mpv}(A_k, \sigma) && [\text{trace_smul}] \end{aligned}$$

The Σ -type trick:

Block-diagonal uses index type $\bigsqcup_k \text{Fin}(D_k)$, not

$\text{Fin}(\sum_k D_k)$. Multiplication is *definitional* via

`blockDiagonal' _mul` — no sub-range arithmetic. Reindex to

$\text{Fin}(\sum D_k)$ only at the end, then immediately remove via

`trace_reindex`.

Design payoff: Avoids >100 lines of `Fin` lemmas.

Why 285 lines?

Each step: separate lemma, induction on word length.

<code>evalWord_blockDiagonal'</code>	20
<code>evalWord_blockDiag' _smul</code>	35
<code>trace_reindex</code>	12
<code>evalWord_reindex</code>	30
<code>mpv_toTensor_eq_sum</code>	65
<code>GaugePhaseEquiv.prop..</code>	55

Scalars accumulate via `smul_smul`, `pow_succ'`.